

.NET 11 · RELEASED 10 NOVEMBER 2026

.NET 11: What's new

C# 15, a faster runtime, handier libraries and a quicker SDK, explained with **code you can run**.

C# 15
new language version

2 years
of support (STS)

18
runnable examples

AGENDA

What we'll cover

1

The release

Support dates, and who should upgrade.

2

C# 15

Unions, closed hierarchies, `with(...)`, extension indexers, labeled loops.

3

Libraries

Process, LINQ joins, JSON unions, Zstandard, async validation, decimals.

4

Runtime

Runtime Async, JIT wins you get for free, new CPU baseline.

5

SDK & tooling

File-based apps, a faster CLI, better `dotnet test`.

6

Upgrade

Ecosystem changes and a 4-step upgrade checklist.

THE RELEASE

Supported versions after 10 November 2026

VERSION	TYPE	RELEASED	SUPPORT ENDS
.NET 11	STS	10 Nov 2026	**14 Nov 2028**
.NET 10	LTS	11 Nov 2025	**14 Nov 2028**
.NET 9	STS	12 Nov 2024	**ended 10 Nov 2026**
.NET 8	LTS	14 Nov 2023	**ended 10 Nov 2026**

LTS = Long-Term Support: 3 years.

STS = Standard-Term Support: 2 years.

So .NET 10 and .NET 11 are both supported until **November 2028**.

Still on .NET 8 or 9? You no longer get security fixes. Move to 10 (LTS) or 11 (STS).

PART

1

C# 15

The default language version for net11.0. No preview flag needed (except memory safety).

IN THIS PART

Union types	01, 02
Closed hierarchies	03
Collection expression arguments	04
Extension indexers	05
Labeled break and continue	06
Memory safety (preview)	07

C# 15 · UNION TYPES

One value, one of a fixed set of types

```
// Implicit conversion: a Dog is a Pet
Pet pet = new Dog("Rex");
Console.WriteLine(Describe(pet));

pet = new Cat("Misty");
Console.WriteLine(Describe(pet));

// Every case is handled, so no "default" arm
static string Describe(Pet pet) => pet switch
{
    Dog d  => $"{d.Name} barks",
    Cat c  => $"{c.Name} meows",
    Bird b => $"{b.Name} sings",
};

public record Cat(string Name);
public record Dog(string Name);
public record Bird(string Name);

// A Pet is exactly one of: Cat, Dog or Bird
public union Pet(Cat, Dog, Bird);
```

`union Pet(Cat, Dog, Bird)` : a Pet is **exactly one** of these three.

Each case converts automatically: `Pet pet = new Dog("Rex")` .

The compiler checks that your `switch` handles **every** case: no `default` arm needed.

Add a fourth case later? Every switch that misses it gets a compiler warning.

OUTPUT

```
$ dotnet run 01-union-types.cs
Rex barks
Misty meows
```

Why it matters: success OR error, checked by the compiler

BEFORE: BASE CLASS

```
abstract record Result;
record Ok(int Value) : Result;
record Error(string Message) : Result;

string Show(Result r) => r switch
{
    Ok ok     => $"OK: {ok.Value}",
    Error err => $"Error: {err.Message}",
    _ => throw new UnreachableException(),
};
```

Before: anyone can add another subclass, so the compiler insists on a `_ =>` fallback.

C# 15: UNION

```
record Ok(int Value);
record Error(string Message);
union Result(Ok, Error);

string Show(Result r) => r switch
{
    Ok ok     => $"OK: {ok.Value}",
    Error err => $"Error: {err.Message}",
};
```

Now: the set of cases is fixed. Ok and Error don't even need a common base class.

C# 15 · CLOSED HIERARCHIES

A class family nobody else can extend

```
Console.WriteLine(Describe(new Closed()));
Console.WriteLine(Describe(new Open(40)));

// Every direct child of GateState is handled, so no default arm.
static string Describe(GateState state) => state switch
{
    Closed => "closed",
    Open(var percent) => $"{percent}% open",
};

// "closed": only this assembly may derive from GateState.
public closed record class GateState;
public record class Closed : GateState;
public record class Open(float Percent) : GateState;
```

> `closed`: only **this assembly** can derive from GateState.

> So the compiler knows **all** children and the switch needs no `default`.

> A closed class is automatically `abstract`.

Unions combine types that already exist. Closed hierarchies are a family you design with a shared base.

OUTPUT

```
$ dotnet run 03-closed-hierarchies.cs
closed
40% open
```

Which one should I use?

SITUATION	USE
The cases already exist and are unrelated (`int`, `string`, a library type)	**union**
You design the types and they share a base (properties, methods)	**closed** hierarchy
A function returns success *or* an error	**union**
A public JSON API contract	**closed** + a type discriminator
You need exhaustive `switch` checks	both ✓

> System.Text.Json writes a union as its **active case only**: `"hello"`, no `$type`.

> That's why Microsoft suggests a closed hierarchy with a discriminator for new API contracts.

C# 15 · COLLECTION EXPRESSION ARGUMENTS

Pass constructor arguments inside [ ]

```
string[] values = ["one", "two", "three"];

// with(...) passes a capacity to List<T>'s constructor
List<string> names = [with(capacity: 10), .. values];
Console.WriteLine($"Capacity: {names.Capacity}");

// with(...) passes a comparer to HashSet<T>'s constructor
HashSet<string> set =
    [with(StringComparer.OrdinalIgnoreCase), "Hi", "HI", "hi"];

// All three are equal when case is ignored
Console.WriteLine($"{set.Count} item");
```

`with(...)` must be the **first** element.

Its arguments go to the collection's constructor: capacity, comparer, …

Before, you had to drop the `[ ]` syntax and write `new HashSet<string>(comparer) { … }`.

OUTPUT

```
$ dotnet run 04-collection-expression-args.cs
Capacity: 10
1 item
```

C# 15 · EXTENSION INDEXERS

Add an indexer to a type you don’t own

```
IEnumerable<int> numbers = Enumerable.Range(1, 10);

// Indexing an IEnumerable<int>, thanks to the extension
Console.WriteLine(numbers[2]);

public static class SequenceIndexer
{
    extension(IEnumerable<int> sequence)
    {
        public int this[int index] => sequence.ElementAt(index);
    }
}
```

> C# 14 introduced `extension` blocks (extension properties).

> C# 15 lets them declare **indexers**: `this[int index]`.

> The receiver needs a name (here `sequence`), because indexers are instance members.

ElementAt walks the sequence: fine for demos, slow for big lists.

OUTPUT

```
$ dotnet run 05-extension-indexers.cs
3
```

C# 15 · LABELED BREAK AND CONTINUE

Leave nested loops without flags or goto

BEFORE: A FLAG

```
bool found = false;
for (int row = 0; row < grid.Length; row++)
{
    for (int col = 0; col < grid[row].Length; col++)
    {
        if (grid[row][col] == 99)
        {
            found = true;
            break;          // only the inner loop
        }
    }
    if (found) break;       // and again here
}
```

C# 15: LABELS

```
outer: for (int row = 0; row < grid.Length; row++)
{
    for (int col = 0; col < grid[row].Length; col++)
    {
        if (grid[row][col] == -1)
            continue outer; // next row

        if (grid[row][col] == 99)
            break outer;    // leave both loops
    }
}
```

Put a label on the loop, then `break outer` or `continue outer`.

Example 06 output: `1 2 5 6 7 8 9`

C# 15 · MEMORY SAFETY (PREVIEW)

unsafe marks where memory is actually read

```
#:property LangVersion=preview
#:property AllowUnsafeBlocks=true

int number = 42;

// Taking an address: no "unsafe" needed any more
int* pointer = &number;

// Reading the memory: still needs "unsafe"
int value = unsafe(*pointer);
Console.WriteLine(value);
```

Preview: needs `LangVersion=preview`.

Taking an address (`&number`) is safe now: nothing is read yet.

Reading through the pointer is still `unsafe`. New: `unsafe(expr)` for one expression.

Goal: reviewers find every risky memory access by searching for **unsafe**.

OUTPUT

```
$ dotnet run 07-memory-safety.cs
42
```

PART

2

Libraries

New APIs that remove boilerplate you write every week.

IN THIS PART

Process: run and capture	01
LINQ FullJoin and tuple Join	02
EqualityComparer.Create, generic Random	03
JSON unions and JSON Lines	04
StringStream and Zstandard	05
Async validation	06
Decimal32 / 64 / 128	07

LIBRARIES · PROCESS

Run a command and read its output

BEFORE: 10 LINES

```
var psi = new ProcessStartInfo("dotnet", "--version")
{
    RedirectStandardOutput = true,
    RedirectStandardError  = true,
};
using var p = Process.Start(psi)!;
string output = await p.StandardOutput.ReadToEndAsync();
await p.WaitForExitAsync();   // forget this: wrong code
Console.WriteLine(output);
Console.WriteLine(p.ExitCode);
```

.NET 11: 1 CALL

```
ProcessTextOutput result =
    await Process.RunAndCaptureTextAsync(
        "dotnet", ["--version"]);

Console.WriteLine(result.StandardOutput);
Console.WriteLine(result.ExitStatus.ExitCode);
```

> Output of example 01: `Output: 11.0.100 · Exit code: 0`

> Also new: `Process.Run`, `StartAndForget`, `KillOnParentExit` (Windows), `Signal(PosixSignal)`.

FullJoin, and Join that returns tuples

```
(int Id, string Name, string? Category)[] products =
    [(1, "Laptop", "Electronics"), (2, "Cable", null)];
(string Name, string Text)[] categories =
    [("Electronics", "Devices"), ("Furniture", "Desks")];

// FullJoin: both sides, paired where they match
var full = products.FullJoin(categories,
    p => p.Category, c => c.Name);
foreach (var (p, c) in full)
    Console.WriteLine($"{p.Name ?? "-"} | {c.Text ?? "-"}");

// Join without a result selector returns tuples
(int ProductId, int Qty)[] orders = [(1, 3), (1, 1)];
var joined = products.Join(orders, p => p.Id, o => o.ProductId);
foreach (var (p, o) in joined)
    Console.WriteLine($"{p.Name}: qty {o.Qty}");
```

FullJoin: every item from **both** sides, `null` where there's no match.

Join without a result selector gives you `(outer, inner)` tuples to deconstruct.

LeftJoin/RightJoin came in .NET 10. EF Core 11 turns FullJoin into SQL **FULL OUTER JOIN**.

OUTPUT

```
$ dotnet run 02-linq-joins.cs
Laptop | Devices
Cable | -
- | Desks
Laptop: qty 3
Laptop: qty 1
```

LIBRARIES · SMALL HELPERS

Comparers from a lambda, random numbers of any type

```
// Compare people by name only: no custom comparer class
var byName = EqualityComparer<Person>.Create(p => p.Name);

var people = new HashSet<Person>(byName)
{
    new("Alice", 30),
    new("Bob", 25),
    new("Alice", 40),   // same name: not added
};
Console.WriteLine($"People: {people.Count}");

// Random numbers of any numeric type
long l = Random.Shared.NextInteger<long>(0, 100);
Half h = Random.Shared.NextBinaryFloat<Half>();
Console.WriteLine($"long={l}, Half={h}");

record Person(string Name, int Age);
```

`EqualityComparer<T>.Create(p => p.Name)` : compare by a key, no custom class.

`NextInteger<T>` : any integer type.
`NextBinaryFloat<T>` : float, double, Half.

The random numbers differ on every run.

OUTPUT

```
$ dotnet run 03-comparer-and-random.cs
People: 2
long=17, Half=0.874
```

Unions and JSON Lines

```
// File-based apps use Native AOT, which turns off reflection JSON:
#:property PublishAot=false
using System.IO.Pipelines;
using System.Text.Json;

// 1) A union is written as its active case: no wrapper
Reading reading = "hello";
Console.WriteLine(JsonSerializer.Serialize(reading));
reading = 42;
Console.WriteLine(JsonSerializer.Serialize(reading));

// 2) JSON Lines: one value per line
var pipe = PipeWriter.Create(Console.OpenStandardOutput());
await JsonSerializer.SerializeAsyncEnumerable(
    pipe, AsyncEnumerable.Range(1, 3), topLevelValues: true);
await pipe.CompleteAsync();

public union Reading(int, string);
```

> A union is serialized as its **active case**: no wrapper object, no `$type`.

> `topLevelValues: true` writes **JSON Lines**: one value per line.

> JSON Lines suits logs and streams: read and write one line at a time.

OUTPUT

```
$ dotnet run 04-json-unions-and-jsonl.cs
"hello"
42
1
2
3
```

No-copy streams and built-in Zstandard

```
using System.IO.Compression;
using System.Text;

string text = string.Concat(Enumerable.Repeat("dotnet ", 1000));

// StringStream: a string as a Stream, no byte[] copy
using var source = new StringStream(text, Encoding.UTF8);

// Zstandard: built in, next to GZip and Brotli
using var packed = new MemoryStream();
using (var zstd = new ZstandardStream(packed,
           CompressionMode.Compress, leaveOpen: true))
{
    source.CopyTo(zstd);
}
Console.WriteLine($"Before: {text.Length:N0} bytes");
Console.WriteLine($"After:  {packed.Length:N0} bytes");
```

StringStream, **ReadOnlyMemoryStream**, **WritableMemoryStream**, **ReadOnlySequenceStream**: wrap data without copying it into a MemoryStream.

ZstandardStream is now in System.IO.Compression, next to GZip and Brotli.

Zstandard: fast, with good compression; widely used for logs and backups.

OUTPUT

```
$ dotnet run 05-streams-and-zstandard.cs
Before: 7,000 bytes
After:  24 bytes
```

LIBRARIES · ASYNC VALIDATION

Validation that can await a database or API

```
using System.ComponentModel.DataAnnotations;

var invoice = new Invoice("DE000");
var errors = new List<ValidationResult>();
bool ok = await Validator.TryValidateObjectAsync(
    invoice, new ValidationContext(invoice), errors, true);
Console.WriteLine(ok ? "Valid" : errors[0].ErrorMessage);

record Invoice([property: VatRegistered] string VatNumber);
class VatRegisteredAttribute : AsyncValidationAttribute
{
    protected override async Task<ValidationResult?> IsValidAsync(
        object? value, ValidationContext ctx, CancellationToken ct)
    {
        await Task.Delay(100, ct);   // e.g. ask the tax office
        return value is "DE123" ? ValidationResult.Success
            : new ValidationResult("VAT number not registered.");
    }
    protected override ValidationResult? IsValid(object? value,
        ValidationContext ctx) => throw new NotSupportedException();
}
```

> Before: `IsValid` was synchronous, so a database check blocked with `.Result`.

> Now: derive from **AsyncValidationAttribute** and put the work in **IsValidAsync**.

> Run it with `Validator.TryValidateObjectAsync`. ASP.NET Core 11 uses it too.

> The sync `IsValid` must still be overridden, but it's never called.

OUTPUT

```
$ dotnet run 06-async-validation.cs
VAT number not registered.
```

LIBRARIES · NUMERICS

Exact decimal math: Decimal32, Decimal64, Decimal128

```
using System.Numerics;

double d = 0.1 + 0.2;
Decimal64 x = Decimal64.Parse("0.1") + Decimal64.Parse("0.2");

Console.WriteLine($"double:    {d:R}");
Console.WriteLine($"Decimal64: {x}");
```

> `double` is binary: 0.1 + 0.2 is **not** exactly 0.3.

> The new types follow the **IEEE 754 decimal** standard, so 0.1 + 0.2 = 0.3.

> Use them to exchange data with other IEEE systems; `decimal` is still fine for money.

OUTPUT

```
$ dotnet run 07-decimal-types.cs
double:    0.30000000000000004
Decimal64: 0.3
```

LIBRARIES · ALSO NEW

More additions worth knowing

AREA	WHAT'S NEW
Compression	ZIP entries are **CRC32-checked**: corrupt files now throw `InvalidDataException`
Base64	`EncodeToString`, `DecodeFromChars`, UTF-8 and span versions
Security	`X25519DiffieHellman` key exchange, AES Key Wrap, experimental ML-KEM / HPKE
Networking	Typed DNS lookups (`DnsResolver`: SRV, MX, TXT), HTTP request compression
Observability	Built-in OpenTelemetry metrics for `MemoryCache`
Console	`FORCE_COLOR=1` keeps colours when output is redirected

PART

3

Runtime

Most of this you get for free: just retarget to net11.0.

IN THIS PART

Runtime Async	01, 02
JIT: faster with no code changes	-
SIMD lane APIs	03
New minimum CPU requirements	-

RUNTIME · RUNTIME ASYNC

The runtime runs async methods itself

COMPILER STATE MACHINES: 13 FRAMES

```
at InnerAsync()
at AsyncMethodBuilderCore.Start(...)
at InnerAsync()
at MiddleAsync()
at AsyncMethodBuilderCore.Start(...)
at MiddleAsync()
at OuterAsync()
at AsyncMethodBuilderCore.Start(...)
at OuterAsync()
at <Main>$(args)
at AsyncMethodBuilderCore.Start(...)
at <Main>$(args)
at Main(args)
```

RUNTIME ASYNC: 5 FRAMES

```
at InnerAsync()
at MiddleAsync()
at OuterAsync()
at <Main>$(args)
at Main(args)
```

Same program (examples 01 and 02): Outer → Middle → Inner, then print `new StackTrace()`.

Turn it on: `<Features>runtime-async=on</Features>`. Fewer frames: clearer debugging and profiling, less overhead.

RUNTIME · JIT

Faster code, no changes needed

IMPROVEMENT	CODE THAT GETS FASTER
Fewer bounds checks	`values[^1]`, `if (!span.IsEmpty && span[0] == x)`
Constant folding	`role == "Admin"` with a known role becomes true/false
Devirtualization	generic virtual methods, default interface methods
Cheaper async	no ExecutionContext copy when you don't use AsyncLocal
Faster math	`Math.BigMul` on x64 is one MUL instruction
ReadyToRun	default comparers precompiled: up to **20×** faster collection ops
Linux	`Guid.NewGuid()` about **12%** faster

Measure your own app: run your benchmarks on .NET 10 and .NET 11.

RUNTIME · SIMD

Build and shuffle vector lanes

```
using System.Runtime.Intrinsics;

// Start at 1, multiply by 2 for each lane: 1, 2, 4, 8
var powers = Vector128.CreateGeometricSequence(1, 2);
Console.WriteLine(powers);

// Interleave two vectors, lane by lane
var a = Vector128.Create(1, 2, 3, 4);
var b = Vector128.Create(10, 20, 30, 40);
var (lower, upper) = Vector128.Zip(a, b);
Console.WriteLine($"{lower} {upper}");
```

- `CreateGeometricSequence(1, 2)` : 1, 2, 4, 8.
- `Zip` interleaves two vectors lane by lane; `Unzip` splits them again.
- Works on `Vector64/128/256/512` and `Vector<T>`. Also: hardware FP16 for `Half`.

OUTPUT

```
$ dotnet run 03-simd-lanes.cs
<1, 2, 4, 8>
<1, 10, 2, 20> <3, 30, 4, 40>
```

RUNTIME · HARDWARE

New minimum CPU requirements

CPU	.NET 10	.NET 11
x64 (Windows, Linux, macOS)	x86-64-v1	**x86-64-v2** (adds SSE4.2, POPCNT)
Arm64 on Windows	armv8.0-a	**armv8.0-a + LSE**
Arm64 on Linux / Apple	unchanged	unchanged (Raspberry Pi still works)

> Every Intel and AMD CPU still supported by its vendor qualifies (older ones went out of support around 2013).

> Windows 11 already requires this level.

On an older CPU, .NET 11 stops with: "The current CPU is missing one or more of the baseline instruction sets."

PART

4

SDK & tooling

A faster CLI, scripts that grow up, and better tests.

IN THIS PART

File-based apps: #:include	app.cs
dotnet run -e	app.cs
A faster dotnet CLI	-
dotnet test and other commands	-

SDK · FILE-BASED APPS

One-file apps can now span several files

```
#:include helpers.cs
#:include models/customer.cs

var customer = new Customer("Vishal", "Augsburg");
var greeting = Environment.GetEnvironmentVariable("GREETING");

Console.WriteLine(Helpers.Format(greeting ?? "Hello", customer));
```

> `dotnet run app.cs`: no .csproj needed (since .NET 10).

> New: `#:include helpers.cs` pulls in other files, even a `.dll`.

> New: `dotnet run -e KEY=VALUE` passes environment variables.

> Without `-e`: `Hello, Vishal from Augsburg!`

OUTPUT

```
$ dotnet run -e GREETING=Hallo app.cs
Hallo, Vishal from Augsburg!
```

SDK · SPEED

A faster dotnet CLI, out of the box

Native AOT

The `dotnet` CLI starts natively. Tools like `dotnet ef` skip 600–700 ms of startup.

On

The MSBuild server is on by default: a warm worker between builds.

−18%

Linux SDK tarball: 230 → 189 MB (deb/rpm: −26%).

10–13%

Faster project evaluation with many files (new wildcard matching).

> Nothing to configure: install the SDK and you get all four.

Opt out if needed: **DOTNET_CLI_ENABLEAOT=false**, **DOTNET_CLI_USE_MSBUILD_SERVER=false**.

SDK · COMMANDS

New options worth trying

```
# Tests: stop after 90 s, or after 5 failures
$ dotnet test --timeout 90s
$ dotnet test --maximum-failed-tests 5

# Solution filters from the CLI
$ dotnet new slnf --parent-solution App.slnx --name Tests
$ dotnet sln Tests.slnf add tests/Tests/Tests.csproj

# xUnit v3 template on Microsoft.Testing.Platform
$ dotnet new xunit --xunit-version v3

# Only use NuGet packages published 24+ hours ago
$ dotnet nuget update source nuget.org --min-publish-age-hours 24
```

`dotnet test` also shows tests that are **running right now**, and supports Android and iOS devices.

Solution filters (.slnf): load only part of a big solution.

Minimum publish age protects you from a freshly published malicious package.

BEYOND THE CORE

What changes in the frameworks

FRAMEWORK	HIGHLIGHTS
ASP.NET Core 11	OpenAPI 3.2, async validation, unions in Minimal APIs, MVC, SignalR, Blazor
EF Core 11	Translates FullJoin, MaxBy, MinBy; fewer joins in includes. **Breaking:** SQL Server compatibility level 160 by default
.NET MAUI	CoreCLR replaces Mono on Android, iOS and Mac Catalyst (opt back: `UseMonoRuntime`)
AI	MCP C# SDK 2.0, `dotnet new mcpserver`, Blazor AI components (experimental)

> EF Core 11 requires .NET 11.

> On SQL Server older than 2022, set `UseCompatibilityLevel` explicitly.

UPGRADE

Four steps to .NET 11

```
// 1. Install the SDK (11.0.100) and pin it in global.json
{ "sdk": { "version": "11.0.100" } }

// 2. Retarget every project (.csproj)
<TargetFramework>net11.0</TargetFramework>

// 3. Update packages: Microsoft.* and EF Core to 11.x
dotnet package update

// 4. Build, run the tests, read the warnings
dotnet build
dotnet test
```

Check the breaking changes before step 4:

- CPU baseline x86-64-v2 (old servers / VMs)
- ZIP files now CRC32-checked
- EF Core: SQL Server compatibility 160
- MAUI: CoreCLR instead of Mono

C# 15 comes automatically with net11.0. No LangVersion change needed.

TRY IT YOURSELF

Every example in this talk runs with one command

```
$ git clone https://github.com/CodeMitVishal/dotnet11-whats-new
$ cd dotnet11-whats-new/examples/01-csharp15
$ dotnet run 01-union-types.cs

# Run every example and refresh the slide outputs
$ python run_examples.py
```

> Needs the **.NET 11 SDK** (11.0.100 or later).

> 18 small apps: C# 15, libraries, runtime and SDK.

> No project files: each `.cs` file is a complete app.



THANK YOU

Thanks for watching!

Questions? Open an issue in the repo or ask in the comments.

youtube.com/@CodeMitVishal

github.com/CodeMitVishal/dotnet11-whats-new

learn.microsoft.com/dotnet/core/whats-new/dotnet-11/overview